Homework 3 Solutions

Biostatistics 735/794

Alexander McLain

2025-11-06

Contents


```
# Core
library(tidyverse)
library(rsample)
library(yardstick)
library(knitr)
library(kableExtra)
library(tidyverse)
library(tidymodels)
```

```r
# Specific models & utilities
library(caret)        # for naive Bayes via caret::train
library(pROC)    # AUC/ROC helper (yardstick also used)
library(MASS)         # birthwt
library(mlbench)      # Heart, BreastCancer
library(rpart)        # CART
library(rpart.plot)   # tree plots
library(ranger)       # fast RF backend
library(randomForest) # alternative RF
library(e1071)        # SVM
library(nnet)         # neural net
library(pdp)          # partial dependence
library(vip)          # variable importance plots


# Small helper: nice tables
kbl2 <- function(x, ...) kable(x, align = "lrrrrrr", ...) %>% kable_styling(full_width = FALSE)
```

A helper to compute the metrics we've discussed (AUC is threshold-free; others use a threshold, default 0.5). Note that I've added a warning if the AUC is less than 0.5.

```r
eval_metrics <- function(truth, prob_pos, level_vals = NA, thr = 0.5) {
  if(is.na(level_vals)){
    if(!is.factor(truth)){
      if(identical(sort(unique(truth)), c(0,1))){
        level_vals <- c("0", "1")
      }else{
        stop("Truth must be a factor, binary 0/1, or the levels need to be given")
      }
    }else{
      level_vals = levels(truth)
    }
  }
  if(!is.factor(truth)){truth <- factor(truth, levels = level_vals)}
  cls   <- factor(ifelse(prob_pos <= thr, level_vals[1],level_vals[2]), levels = level_vals)
  res <- tibble(
    AUC  = roc_auc_vec(truth, prob_pos, event_level = "second"),
    Sens = sens_vec(truth, cls,        event_level = "second"),
    Spec = spec_vec(truth, cls,        event_level = "second"),
    PPV  = ppv_vec (truth, cls,        event_level = "second"),
    NPV  = npv_vec (truth, cls,        event_level = "second"),
    Acc  = accuracy_vec(truth, cls)
  )
  if(res$AUC < 0.5){warning(paste("The AUC is less than 0.5. Check the coding consistency. \n
                                  High probability currently means the subject is more likely", level_va
  return(res)
}
```

1. Naive Bayes on dataset from UCI (Cleveland version)

Each row is a patient. Outcome `Diagnosis` is **B**enign or **M**alignant. Predictors are 30 numeric features derived from FNA images.

1(a) Explore the dataset

```
# --- Read the dataset from UCI (Cleveland version) ---
url <- "https://archive.ics.uci.edu/ml/machine-learning-databases/heart-disease/processed.cleveland.data"

heart_raw <- read.csv(url, header = FALSE, na.strings = "?")

# --- Assign variable names based on UCI documentation ---
colnames(heart_raw) <- c(
  "age",        # age in years
  "sex",        # sex (1 = male; 0 = female)
  "cp",         # chest pain type (1-4)
  "trestbps",   # resting blood pressure (mm Hg)
  "chol",       # serum cholesterol (mg/dl)
  "fbs",        # fasting blood sugar > 120 mg/dl (1 = true; 0 = false)
  "restecg",    # resting electrocardiographic results (0-2)
  "thalach",    # max heart rate achieved
  "exang",      # exercise induced angina (1 = yes; 0 = no)
  "oldpeak",    # ST depression induced by exercise relative to rest
  "slope",      # slope of peak exercise ST segment (1-3)
  "ca",         # number of major vessels (0-3) colored by fluoroscopy
  "thal",       # 3 = normal; 6 = fixed defect; 7 = reversible defect
  "target"      # 0 = no disease; 1-4 = presence of heart disease
)


# --- Convert columns to appropriate types ---
heart_clean <- heart_raw %>%
  mutate(
    sex      = factor(sex, levels = c(0, 1), labels = c("Female", "Male")),
    cp       = factor(cp, levels = 1:4, labels = c("Typical", "Atypical", "Non-anginal", "Asymptomatic")),
    fbs      = factor(fbs, levels = c(0, 1), labels = c("False", "True")),
    restecg  = factor(restecg, levels = 0:2, labels = c("Normal", "ST-T abnormality", "LVH")),
    exang    = factor(exang, levels = c(0, 1), labels = c("No", "Yes")),
    slope    = factor(slope, levels = 1:3, labels = c("Upsloping", "Flat", "Downsloping")),
    ca       = as.numeric(ca),
    thal     = factor(thal, levels = c(3, 6, 7), labels = c("Normal", "Fixed", "Reversible")),
    target   = factor(ifelse(target == 0, "NoDisease", "Disease"))
  )

# --- Handle missing values ---
# Count missing (not much)
colSums(is.na(heart_clean))
```

```
##      age      sex       cp trestbps     chol      fbs  restecg  thalach
##        0        0        0        0        0        0        0        0
##    exang  oldpeak    slope       ca     thal   target
##        0        0        0        4        2        0
```

```
# Drop rows with missing values (most analyses do)
heart_final <- heart_clean %>% drop_na()
```

1(b) Train/Test Split

```
set.seed(42)
split <- initial_split(heart_final, prop = 0.7)
train <- training(split)
test  <- testing(split)

nrow(train); nrow(test)
```

```
## [1] 207
```

```
## [1] 90
```

1(c) Fit Naive Bayes (Caret)

```
train_control <- trainControl(
  method = "cv",
  number = 10,
  classProbs = TRUE,
  summaryFunction = twoClassSummary)

tune_grid <- expand.grid(
  laplace = c(0, 0.5, 1),
  usekernel = c(TRUE, FALSE),
  adjust = c(0.5, 1, 2)
)

set.seed(123)
fit_nb <- train(
  target ~ .,
  data = train,
  method = "naive_bayes",
  metric = "ROC",
  trControl = train_control,
  tuneGrid = tune_grid
)

fit_nb$bestTune
```

```
##   laplace usekernel adjust
## 6       0      TRUE      2
```

1(d) Evaluate on Test Set

```
nb_pred_prob <- predict(fit_nb, newdata = test, type = "prob")[, "NoDisease"]
nb_pred_cls  <- predict(fit_nb, newdata = test)

nb_results <- eval_metrics(test$target, nb_pred_prob, thr = 0.5)
nb_results %>% kbl2(digits = 3)
```

AUC	Sens	Spec	PPV	NPV	Acc
0.897	0.695	0.839	0.891	0.591	0.744

1(e) Re-fit with Binned Variables

```r
# Example: bin Age and Cholesterol
heart_binned <- heart_final %>%
  mutate(
    AgeBin  = cut(age, breaks = c(0, 45, 55, 65, Inf), right = FALSE),
    CholBin = cut(chol, breaks = quantile(chol, probs = seq(0, 1, 0.25), na.rm = TRUE), include.lowest =
  ) %>%
  dplyr::select(-age, -chol)

split_h2 <- initial_split(heart_binned, prop = 0.7)
h2_tr <- training(split_h2)
h2_te <- testing(split_h2)

nb_fit2 <- caret::train(
  target ~ .,
  data = h2_tr,
  method = "naive_bayes",
  metric = "ROC",
  trControl = train_control,
  tuneGrid = tune_grid
)

nb2_prob <- predict(nb_fit2, newdata = h2_te, type = "prob")[, "NoDisease"]
nb2_cls  <- predict(nb_fit2, newdata = h2_te)

nb_results2 <- eval_metrics(h2_te$target, nb2_prob, thr = 0.5)

### Compare the results

nb_results %>%
  mutate(Type = "Original", .before = AUC) %>%
  bind_rows(
    nb_results2 %>% mutate(Type = "Binned", .before = AUC)
  ) %>%
  kbl2(digits = 3)
```

Type	AUC	Sens	Spec	PPV	NPV	Acc
Original	0.897	0.695	0.839	0.891	0.591	0.744
Binned	0.925	0.727	0.891	0.865	0.774	0.811

Interpretation: Compare AUC/accuracy/sens/spec between raw vs. binned features; Naive Bayes can benefit when independence assumptions are closer to true after discretization.

1(f) When is Naive Bayes advantageous?

- Works well with mixed data and small samples, robust with many predictors.
- Fast baseline; often competitive when conditional independence is approximately satisfied.
- In public health, good for quick triage models / risk screening where interpretability and speed matter.

2. CART Regression Tree — MASS::birthwt

2(a) Describe Data

```
data(birthwt, package = "MASS")
bw <- birthwt %>% dplyr::select(-low) %>% as_tibble()
glimpse(bw)
```

```
## Rows: 189
## Columns: 9
## $ age   <int> 19, 33, 20, 21, 18, 21, 22, 17, 29, 26, 19, 19, 22, 30, 18, 18, ~
## $ lwt   <int> 182, 155, 105, 108, 107, 124, 118, 103, 123, 113, 95, 150, 95, 1~
## $ race  <int> 2, 3, 1, 1, 1, 3, 1, 3, 1, 1, 3, 3, 3, 3, 1, 1, 2, 1, 3, 1, 3, 1~
## $ smoke <int> 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0~
## $ ptl   <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0~
## $ ht    <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0~
## $ ui    <int> 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1~
## $ ftv   <int> 0, 3, 1, 2, 0, 0, 1, 1, 1, 0, 0, 1, 0, 2, 0, 0, 0, 3, 0, 1, 2, 3~
## $ bwt   <int> 2523, 2551, 2557, 2594, 2600, 2622, 2637, 2637, 2663, 2665, 2722~
```

```
summary(bw$bwt)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     709    2414    2977    2945    3487    4990
```

Outcome: `bwt` (birth weight, grams). Predictors include maternal age, weight, race, smoking status, hypertension history, etc.

2(b) Fit CART Regression Tree

```
tree_fit <- rpart(bwt ~ ., data = bw, method = "anova",
                  control = rpart.control(cp = 0.001, minsplit = 10))
printcp(tree_fit)
```

```
##
## Regression tree:
## rpart(formula = bwt ~ ., data = bw, method = "anova", control = rpart.control(cp = 0.001,
##     minsplit = 10))
##
## Variables actually used in tree construction:
## [1] age   ht    lwt   race  smoke ui
##
## Root node error: 99969656/189 = 528940
##
## n= 189
##
##           CP nsplit rel error  xerror     xstd
## 1  0.0844485      0   1.00000 1.00685 0.099499
## 2  0.0818680      1   0.91555 1.02147 0.099855
## 3  0.0434671      2   0.83368 0.89499 0.093354
## 4  0.0293049      4   0.74675 0.88796 0.088999
## 5  0.0240662      6   0.68814 0.89074 0.088842
## 6  0.0238337      7   0.66407 0.91555 0.090083
## 7  0.0178182      9   0.61641 0.95244 0.093002
## 8  0.0129702     10   0.59859 1.10257 0.101525
```

```
## 9  0.0129541     11   0.58562 1.11659 0.102484
## 10 0.0127575     12   0.57266 1.13966 0.104473
## 11 0.0120921     13   0.55991 1.14719 0.105701
## 12 0.0111578     14   0.54781 1.16524 0.109042
## 13 0.0099946     15   0.53666 1.18625 0.109416
## 14 0.0099247     16   0.52666 1.18638 0.109619
## 15 0.0093123     17   0.51674 1.18115 0.109635
## 16 0.0086887     18   0.50742 1.18351 0.107179
## 17 0.0082723     19   0.49874 1.20320 0.109614
## 18 0.0078314     21   0.48219 1.20633 0.109361
## 19 0.0072483     25   0.45087 1.20600 0.109265
## 20 0.0067943     29   0.41873 1.21947 0.109604
## 21 0.0049514     31   0.40514 1.23267 0.113528
## 22 0.0048466     33   0.39524 1.25217 0.113988
## 23 0.0036095     34   0.39039 1.24587 0.112925
## 24 0.0010000     35   0.38679 1.23965 0.112587
```

2(c) Cross-Validation for cp

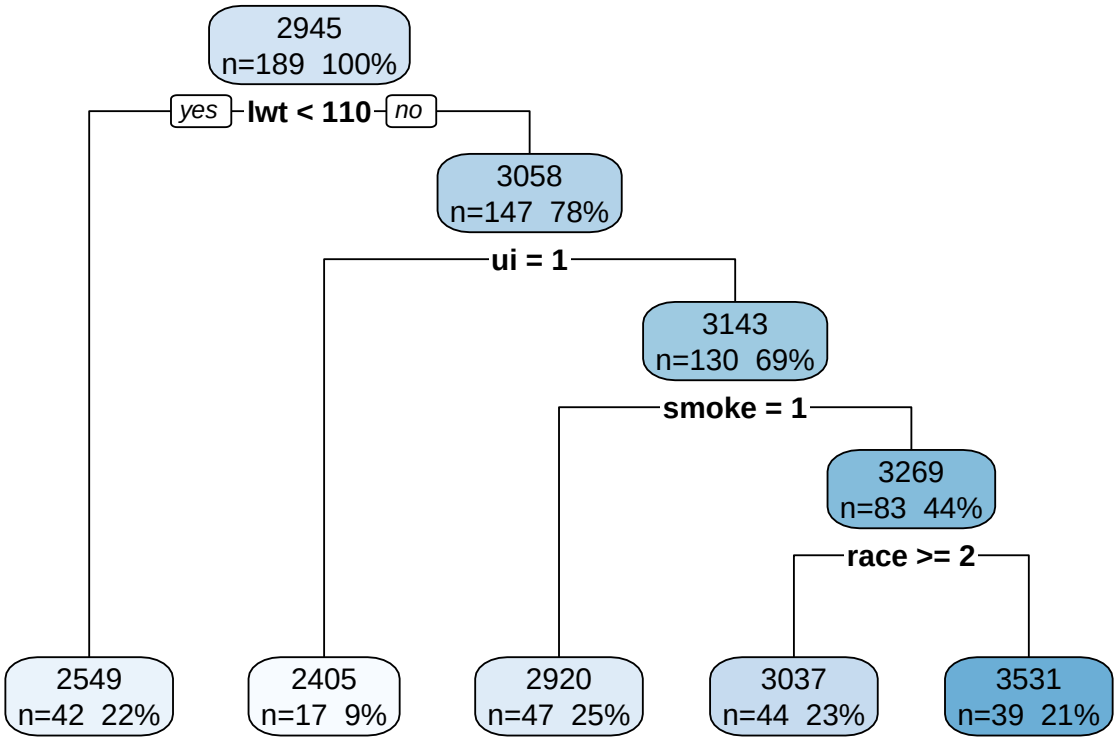
```
best_cp <- tree_fit$cptable[which.min(tree_fit$cptable[,"xerror"]), "CP"]
best_cp
```

```
## [1] 0.02930493
```

```
tree_pruned <- prune(tree_fit, cp = best_cp)
```

2(d) Plot and Interpret Top Splits

```
rpart.plot(tree_pruned, extra = 101)
```



Interpretation (partial): The first split is on `lwt`, separating mothers with `lwt < 110` vs `lwt>= 110`. In context, this suggests mother's with lower weight in pounds at last menstrual period have babies that weigh less. The second split further differentiates for women who had `lwt>= 110`, for those that had presence of uterine irritability, their babies weighed less.

2(e) Compare RMSE to Linear Regression

```
set.seed(2)
split_bw <- initial_split(bw, prop = 0.8, strata = NULL)
bw_tr <- training(split_bw)
bw_te <- testing(split_bw)

# Fit/prune on training data using best_cp from full fit (simple approach)
tree_pruned_te <- predict(prune(rpart(bwt ~ ., data = bw_tr, method="anova",
                                      control = rpart.control(cp = best_cp)), cp = best_cp), newdata = b

lm_fit <- lm(bwt ~ ., data = bw_tr)
lm_pred <- predict(lm_fit, newdata = bw_te)

rmse_tree <- yardstick::rmse_vec(bw_te$bwt, tree_pruned_te)
rmse_lm   <- yardstick::rmse_vec(bw_te$bwt, lm_pred)
c(RMSE_tree = rmse_tree, RMSE_lm = rmse_lm)
```

```
## RMSE_tree   RMSE_lm
##  748.1225  638.0250
```

2(f) Overfitting (Optional)

Deep trees can overfit noise (low training error, poor generalization). CP pruning via CV reduces variance.

---

3. Random Forest & SVM Regression — AirQuality

We will use `datasets::airquality` and predict `Ozone` (continuous).

3(a) Split and Preprocess

```
aq <- airquality %>% as_tibble()

# Simple cleaning: drop rows with all-NA Ozone, impute via recipe for remaining
split_aq <- initial_split(aq %>% drop_na(Ozone), prop = 0.7)
aq_tr <- training(split_aq)
aq_te <- testing(split_aq)

rec_aq <- recipe(Ozone ~ ., data = aq_tr) %>%
  step_impute_median(all_numeric_predictors()) %>%
  step_normalize(all_numeric_predictors())

prep_aq <- prep(rec_aq)
aq_tr_baked <- bake(prep_aq, new_data = aq_tr)
aq_te_baked <- bake(prep_aq, new_data = aq_te)
```

3(b) Fit Random Forest (fixed ntree/mtry)

```
rf_spec <- rand_forest(mtry = 3, trees = 1000, min_n = 5) %>%
  set_engine("ranger", importance = "permutation") %>%
  set_mode("regression")

wf_rf <- workflow() %>% add_model(rf_spec) %>% add_recipe(rec_aq)
fit_rf <- fit(wf_rf, data = aq_tr)
```

3(c) Tune mtry via CV

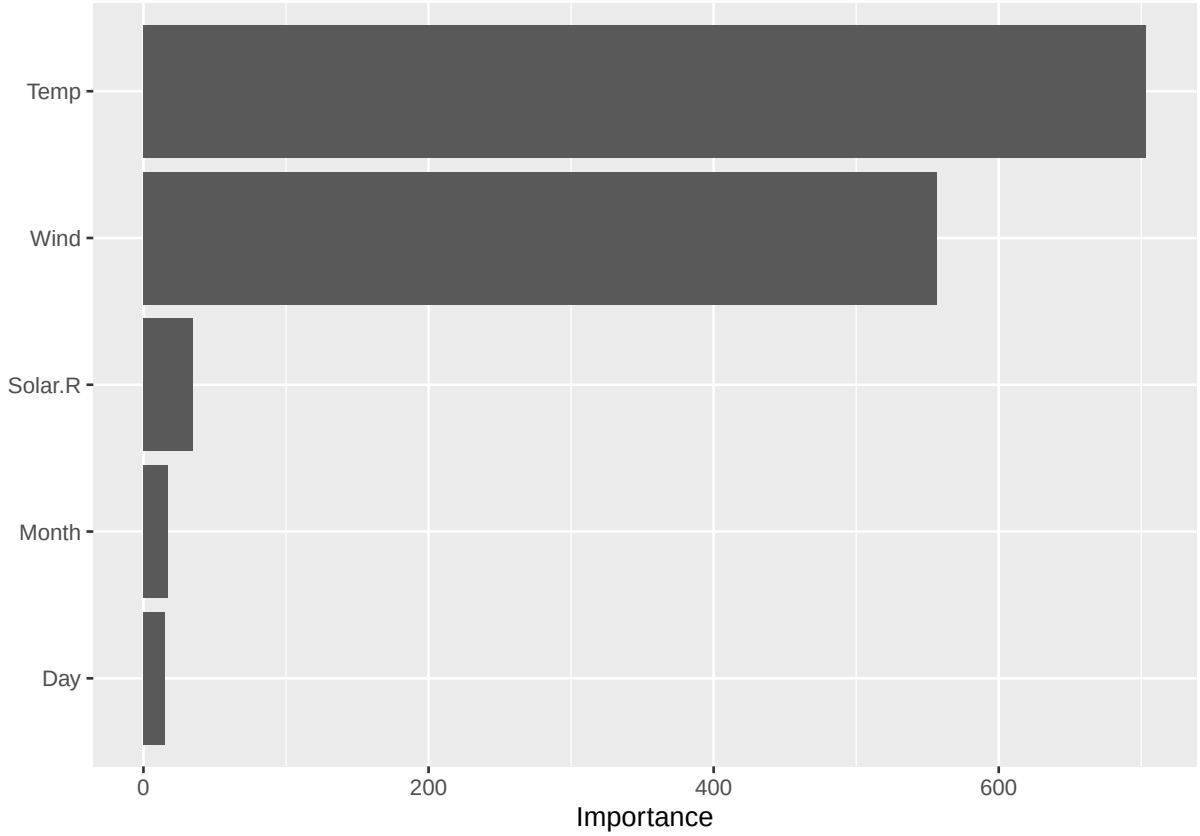
```
set.seed(3)
folds <- vfold_cv(aq_tr, v = 5)
grid <- tibble(mtry = 2:5)

rf_tuned <- tune_grid(
  wf_rf %>% update_model(rand_forest(mtry = tune(), trees = 1000, min_n = 5) %>%
                           set_engine("ranger", importance="permutation") %>%
                           set_mode("regression")),
  resamples = folds,
  grid = grid,
  metrics = metric_set(rmse, rsq)
)

show_best(rf_tuned, metric = "rmse")
```

```
## # A tibble: 4 x 7
##    mtry .metric .estimator  mean     n std_err .config
##   <int> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1     3 rmse    standard    19.2     5    2.02 pre0_mod2_post0
## 2     4 rmse    standard    19.3     5    1.83 pre0_mod3_post0
## 3     2 rmse    standard    19.5     5    2.12 pre0_mod1_post0
## 4     5 rmse    standard    19.6     5    1.87 pre0_mod4_post0
```

```
best <- select_best(rf_tuned, metric = "rmse")
rf_final <- finalize_workflow(wf_rf, best)
fit_rf_final <- fit(rf_final, data = aq_tr)
```

3(d) Variable Importance

```
rf_obj <- extract_fit_parsnip(fit_rf_final)$fit
vip::vip(rf_obj)
```

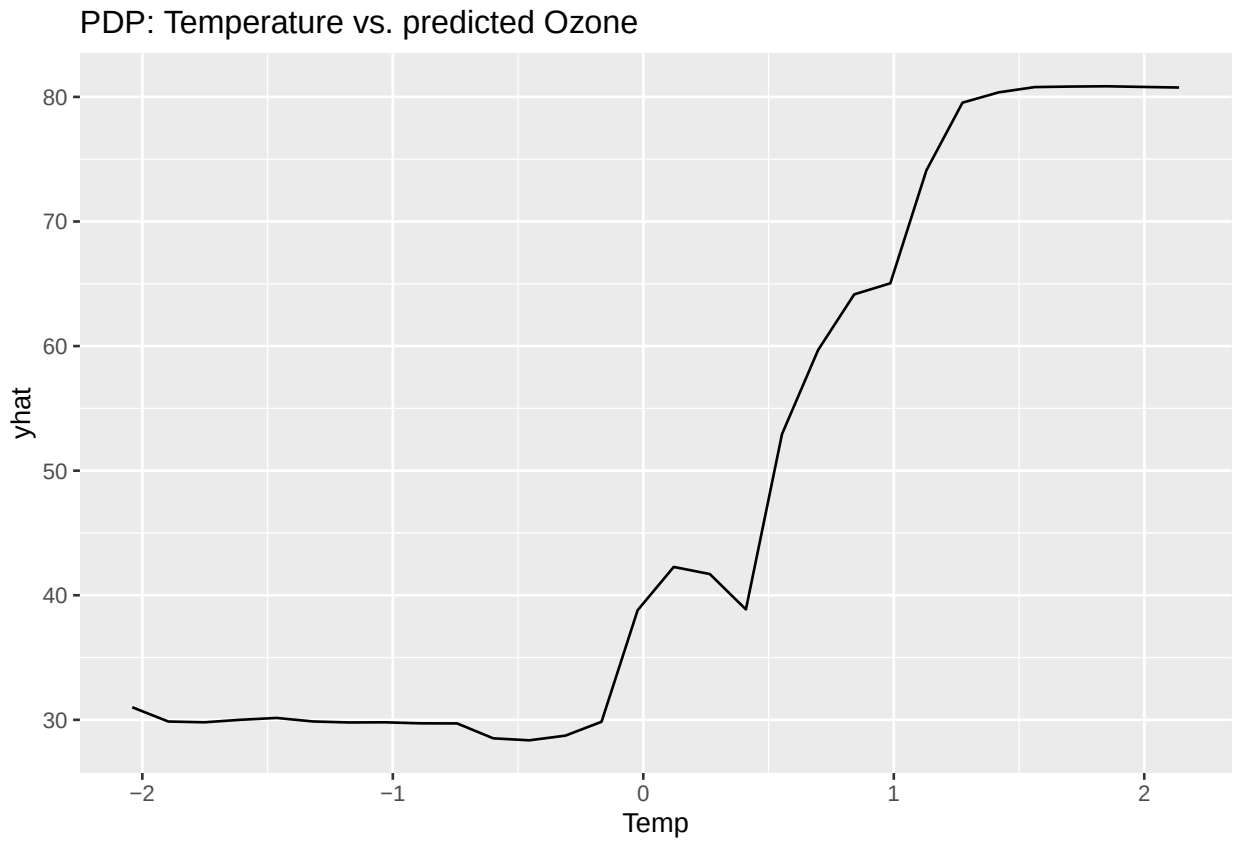


Interpretation: Identify top 3 predictors and provide a short rationale (e.g., `Temp`, `Wind`, `Solar.R` commonly drive ozone levels).

3(e) Partial Dependence Plot

```
pdp::partial(
  object    = rf_obj,
  pred.var = "Temp",
  train     = aq_tr_baked,
```

```
  grid.resolution = 30,
  prob = TRUE
) %>% autoplot() + ggtitle("PDP: Temperature vs. predicted Ozone")
```



3(f) SVM with RBF kernel (tune cost)

```
svm_spec <- svm_rbf(cost = tune(), rbf_sigma = tune()) %>%
  set_engine("kernlab") %>% set_mode("regression")

grid_svm <- grid_regular(cost(), rbf_sigma(), levels = 4)

wf_svm <- workflow() %>% add_model(svm_spec) %>% add_recipe(rec_aq)

set.seed(4)
svm_tuned <- tune_grid(
  wf_svm, resamples = folds, grid = grid_svm,
  metrics = metric_set(rmse, rsq)
)

best_svm <- select_best(svm_tuned, metric = "rmse")
svm_final <- finalize_workflow(wf_svm, best_svm)
fit_svm <- fit(svm_final, data = aq_tr)
```

3(g) Compare Test RMSE

```
rf_pred <- predict(fit_rf_final, new_data = aq_te)$.pred
sv_pred <- predict(fit_svm,      new_data = aq_te)$.pred

c(
  RF_RMSE  = yardstick::rmse_vec(aq_te$Ozone, rf_pred),
  SVM_RMSE = yardstick::rmse_vec(aq_te$Ozone, sv_pred)
)
```

```
##  RF_RMSE SVM_RMSE
## 12.88330 14.54166
```

---

4. Neural Network — BreastCancer (mlbench)

4(a) Preprocess

```
data(BreastCancer, package = "mlbench")
bc <- BreastCancer %>% as_tibble() %>%
  dplyr::select(-Id) %>%
  mutate(across(-Class, ~as.numeric(as.character(.)))) %>%
  mutate(Class = factor(Class)) %>%
  drop_na()

split_bc <- initial_split(bc, prop = 0.8, strata = Class)
bc_tr <- training(split_bc)
bc_te <- testing(split_bc)

rec_bc <- recipe(Class ~ ., data = bc_tr) %>%
  step_normalize(all_numeric_predictors())

prep_bc <- prep(rec_bc)
```

4(b,c) Single Hidden-Layer Neural Net (tune size & decay)

```
nn_spec <- mlp(hidden_units = tune(), penalty = tune(), activation = "relu") %>%
  set_engine("nnet", trace = FALSE) %>%
  set_mode("classification")

grid_nn <- grid_regular(
  hidden_units(range = c(2L, 10L)),
  penalty(range = c(-5, 0)) # For penalization (on log10 scale internally)
)

wf_nn <- workflow() %>% add_model(nn_spec) %>% add_recipe(rec_bc)

set.seed(5)
folds_bc <- vfold_cv(bc_tr, v = 5, strata = Class)

nn_tuned <- tune_grid(
  wf_nn, resamples = folds_bc, grid = grid_nn,
```

```
  metrics = metric_set(roc_auc, accuracy)
)

show_best(nn_tuned, metric = "roc_auc") %>% kbl2(digits = 5)
```

hidden_units	penalty	.metric	.estimator	mean	n	std_err	.config
6	1.00000	roc_auc	binary	0.99595	5	0.00165	pre0_mod6_post0
10	1.00000	roc_auc	binary	0.99595	5	0.00165	pre0_mod9_post0
2	1.00000	roc_auc	binary	0.99581	5	0.00170	pre0_mod3_post0
2	0.00316	roc_auc	binary	0.99103	5	0.00287	pre0_mod2_post0
10	0.00316	roc_auc	binary	0.99052	5	0.00257	pre0_mod8_post0

```
## I'm going to use the model with 2 versus 6 hidden layest:
best_nn <- select_by_one_std_err(nn_tuned, metric = "roc_auc", hidden_units)
nn_final <- finalize_workflow(wf_nn, best_nn)
fit_nn <- fit(nn_final, data = bc_tr)
```

4(d,e) Compare to Logistic Regression and Random Forest

```
# Logistic regression
log_spec <- logistic_reg() %>% set_engine("glm")
wf_log <- workflow() %>% add_model(log_spec) %>% add_recipe(rec_bc)
fit_log <- fit(wf_log, data = bc_tr)

# Random Forest
rf_spec_cls <- rand_forest(mtry = tune(), trees = 1000, min_n = 5) %>%
  set_engine("ranger", importance = "permutation") %>%
  set_mode("classification")
grid_rf <- tibble(mtry = 2:5)

folds_bc2 <- vfold_cv(bc_tr, v = 5, strata = Class)
rf_tuned_cls <- tune_grid(
  workflow() %>% add_model(rf_spec_cls) %>% add_recipe(rec_bc),
  resamples = folds_bc2, grid = grid_rf,
  metrics = metric_set(roc_auc, accuracy)
)
best_rf_cls <- select_best(rf_tuned_cls, metric = "roc_auc")
rf_final_cls <- finalize_workflow(workflow() %>% add_model(rf_spec_cls) %>% add_recipe(rec_bc), best_rf
fit_rf_cls <- fit(rf_final_cls, data = bc_tr)


nn_pr <- predict(fit_nn, bc_te, type = "prob")
log_pr<- predict(fit_log, bc_te, type = "prob")
rf_pr <- predict(fit_rf_cls, bc_te, type = "prob")

nn_results <- eval_metrics(bc_te$Class, nn_pr$.pred_malignant, thr = 0.5) %>%
  mutate(Model = "Neural Nets", .before = AUC)
log_results <- eval_metrics(bc_te$Class, log_pr$.pred_malignant, thr = 0.5) %>%
  mutate(Model = "Logistic", .before = AUC)
rf_results <- eval_metrics(bc_te$Class, rf_pr$.pred_malignant, thr = 0.5) %>%
  mutate(Model = "Random Forests", .before = AUC)
```

```r
nn_results %>%
  bind_rows(log_results) %>%
  bind_rows(rf_results) %>%
  kbl2(digits = 3)
```

Model	AUC	Sens	Spec	PPV	NPV	Acc
Neural Nets	0.994	0.938	0.966	0.938	0.966	0.956
Logistic	0.993	0.917	0.978	0.957	0.956	0.956
Random Forests	0.989	0.938	0.966	0.938	0.966	0.956

4(f) When are neural networks unnecessary?

- When data are low-dimensional, linearly separable, or sample size is modest.
- When interpretability and uncertainty quantification are priorities (logistic regression/trees preferred).
- When simpler models already achieve near-optimal performance (like here).
